

Pràctica 2

2 Les eines matemàtiques

Abans de construir RSA hem implementat les eines matemàtiques que utilitza.

2.1 Màxim comú divisor

El màxim comú divisor (mcd) de dos enters és el nombre enter positiu més gran que els divideix a tots dos. L'algorisme d'Euclides el calcula aplicant successivament la divisió entera fins que el residu és zero, moment en què el divisor actual és el mcd. Com a comprovació, $\text{mcd}(240, 46) = 2$ ja que $240 = 5 \cdot 46 + 10$, $46 = 4 \cdot 10 + 6$, $10 = 1 \cdot 6 + 4$, $6 = 1 \cdot 4 + 2$ i $4 = 2 \cdot 2 + 0$.

2.2 Algorisme d'Euclides estès

Per implementar RSA necessitem una versió més potent: **l'algorisme d'Euclides estès**, que a més de trobar el MCD ens proporciona els coeficients de la **identitat de Bézout**.

Donats dos enters a i b , la identitat de Bézout afirma que existeixen dos enters x i y tals que:

$$ax + by = \text{mcd}(a, b)$$

La funció `euclides_extens(a, b)` retorna la terna (d, x, y) on $d = \text{mcd}(a, b)$ i x, y són els coeficients de Bézout.

L'algorisme funciona de manera recursiva. El cas base és quan $b = 0$: en aquest moment el MCD és a i els coeficients són $x = 1$, $y = 0$ (ja que $a \cdot 1 + 0 \cdot 0 = a$). En el pas recursiu es crida la funció amb $(b, a \bmod b)$ i a partir dels coeficients obtinguts es reconstrueixen els coeficients del nivell anterior mitjançant la relació:

$$x_{\text{actual}} = y_{\text{següent}}, \quad y_{\text{actual}} = x_{\text{següent}} - \lfloor a/b \rfloor \cdot y_{\text{següent}}$$

Verificació: Per a $a = 240$ i $b = 46$ obtenim:

$$240 \cdot (-9) + 46 \cdot 47 = -2160 + 2162 = 2$$

on $2 = \text{mcd}(240, 46)$.

2.3 Invers modular

L'invers modular d' a mòdul n és l'enter x tal que:

$$a \cdot x \equiv 1 \pmod{n}$$

És a dir, el nombre que multiplicat per a dona 1 en aritmètica modular.

Per calcular-lo fem servir directament l'algorisme d'Euclides estès. Si apliquem `euclides_extens(a, n)` obtenim (d, x, y) tal que:

$$a \cdot x + n \cdot y = d$$

Si prenem aquesta expressió mòdul n , el terme $n \cdot y$ desapareix i ens queda:

$$a \cdot x \equiv d \pmod{n}$$

Per tant, si $d = 1$ (és a dir, si a i n són coprimers), llavors x és exactament l'invers modular d' a . Si $d \neq 1$, l'invers **no existeix**. Això passa quan a i n tenen factors comuns, i la funció retorna None per indicar-ho. El resultat final es calcula com $x \% n$ per assegurar-nos que l'invers estigui en el rang $[0, n - 1]$, ja que els coeficients de Bézout poden ser negatius.

Verificació: Per a $a = 17$ i $n = 3120$:

$$17 \cdot 2753 = 46801 = 15 \cdot 3120 + 1 \equiv 1 \pmod{3120}$$

Per tant:

$$17^{-1} \equiv 2753 \pmod{3120}$$

Aquest resultat serà clau en RSA: l'exponent privat d s'obté precisament calculant l'invers modular d' e respecte a $\varphi(n)$.

2.4 Exponenciació modular ràpida

L'objectiu és calcular $a^e \pmod n$ sense calcular mai a^e explícitament, perquè aquest nombre seria astronòmicament gran. L'algorisme escriu e en binari i el recorre bit a bit. A cada pas es quadra la base i es redueix el resultat mòdul n , i quan el bit corresponent és 1 es multiplica l'acumulador per la base i es redueix un altre cop. D'aquesta manera els valors intermedis mai superen n^2 .

Per a l'exemple de l'enunciat, $65^{17} \pmod{3233}$, escrivim $17 = 10001_2$, és a dir $17 = 16 + 1$. El nostre algorisme fa 5 quadratures, una per cada bit, i 2 multiplicacions, les que corresponen als bits que són 1. Són 7 operacions en total i el resultat és 2790, idèntic al que dona `pow(65, 17, 3233)`.

Per què el nombre de multiplicacions és proporcional al nombre de bits de e ? Perquè el bucle recorre exactament els bits de e i a cada bit fa un treball constant, una quadratura i com a molt una multiplicació més. Si e té k bits, el cost és com a molt $2k$ multiplicacions i de mitjana unes $1,5k$, és a dir $O(\log e)$. En canvi, la multiplicació successiva faria $e - 1$ multiplicacions. Per a $e = 65537$, que té 17 bits, el nostre algorisme en fa 19, 17 quadratures i 2 multiplicacions, mentre que l'ingenu en necessitaria 65536. A més, com que es redueix mòdul n a cada pas, els nombres es mantenen petits fins i tot per a exponents de milers de bits.

3 Construïm RSA

3.1 Un primer exemple

Amb $p = 61$, $q = 53$ i $e = 17$ el programa calcula $n = pq = 3233$ i $\varphi(n) = (p - 1)(q - 1) = 3120$, comprova que $\text{mcd}(e, \varphi(n)) = 1$ i obté $d = e^{-1} \pmod{\varphi(n)} = 2753$. La clau pública és $(n, e) = (3233, 17)$ i la privada és $(n, d) = (3233, 2753)$, tal com demanava l'enunciat.

3.2 Xifratge i desxifratge

Fent servir exclusivament la nostra exponenciació modular, per a $m = 65$ obtenim $c = 65^{17} \pmod{3233} = 2790$ i desxifrant $m = 2790^{2753} \pmod{3233}$ recuperem 65. La funció xifra a més rebutja valors que no compleixin $0 \leq m < n$, així que `xifra(3233, 17, 3233)` retorna None.

3.3 Les nostres claus

Hem construït la nostra parella de claus amb dos primers de la llista de l'enunciat, $p = 1009$ i $q = 1013$, i $e = 65537$. Els valors obtinguts són $n = 1022117$, $\varphi(n) = 1008 \cdot 1012 = 1020096$ i $d = 832193$.

El programa rebutja els paràmetres invàlids i retorna None en aquests casos.

- $p = q$, per exemple `genera_claus(1009, 1009, 65537)`
- valors que no són primers, per exemple `genera_claus(1001, 1013, 65537)` amb $1001 = 7 \cdot 11 \cdot 13$
- exponents no coprimers amb $\varphi(n)$, per exemple `genera_claus(1009, 1013, 4)` i `genera_claus(1009, 1013, 1000)`, perquè $\varphi(n)$ és parell

4 Xifrem un text

4.1 Del text als bytes

1. `ord("ú")` retorna 250.
2. UTF-8 utilitza 2 bytes per representar ú, concretament `[195, 186]`.
3. Un caràcter i un byte no són la mateixa cosa. Un caràcter és un símbol amb un punt de codi Unicode, que pot arribar a superar el milió, i un byte només té 8 bits i 256 valors possibles. L'ASCII cabe en un sol byte, però Unicode té més de 150000 caràcters, per això cal una codificació com UTF-8 que converteix cada caràcter en 1, 2, 3 o 4 bytes.
4. Si volem xifrar un fitxer o un missatge real és més natural treballar amb els bytes perquè els fitxers i les transmissions es desen i es transmeten com a seqüències de bytes i UTF-8 és l'estàndard per al text. Si xifréssim els codis `ord()` obtindríem nombres que, en desxifrar-los, no reconstruirien els bytes desats i el fitxer original es corrompria.

4.2 RSA byte a byte

Hem seguit els passos de l'enunciat. El text es converteix a UTF-8, cada byte es xifra independentment amb `xifra`, es guarden els enters xifrats, es desxifren amb `desxifra`, es reconstrueix la seqüència de bytes i es recupera el text amb `.decode("utf-8")`. Amb la nostra clau hem xifrat i recuperat correctament `La criptografia és divertida!`.

RSA només pot treballar amb enters que compleixin $0 \leq m < n$ i un byte pot valdre entre 0 i 255. El valor més gran que pot prendre un byte és 255, per tant cal que n sigui més gran que qualsevol byte, $n > 255$, perquè si no hi hauria bytes que no cabrien en l'interval i no es podrien xifrar. El nostre programa comprova aquesta condició i retorna None si $n \leq 255$, tal com passa amb `xifra_text("Hola!", 17, 200)`.

4.3 Què passa amb les repeticions?

Hem xifrat `AAAAAAAAAA`, `MISSISSIPPI`, `ABRACADABRA` i `AABBAABBAABB` amb la nostra clau. `AAAAAAAAAA` produeix deu vegades el mateix enter, 341568. A `MISSISSIPPI` surten `{368745: 4, 520249: 4, 238996: 2}`, que corresponen a les lletres I, S i P, i el

criptograma complet és 428438, 368745, 520249, 520249, 368745, 520249, 520249, 368745, 238996, 238996, 368745.

1. **Dues aparicions del mateix byte produeixen sempre el mateix criptograma?**
Sí. El xifratge RSA és una funció, $c = m^e \bmod n$, així que el mateix byte sempre dona el mateix resultat.
2. **Es poden detectar patrons en el criptograma?**
Sí. Cada repetició del text en clar es reflecteix com una repetició exacta de l'enter xifrat, tal com mostren els exemples anteriors.
3. **Un observador que no conegui la clau privada podria saber que determinades posicions contenen el mateix byte?**
Sí. Només comparant els enters xifrats es veu quines posicions són iguals, sense conèixer el byte que hi ha darrere.
4. **Quina informació del text original continua sent visible?**
La longitud exacta del missatge en bytes, la posició de tots els bytes iguals entre si i la freqüència de cada byte diferent. Amb aquestes freqüències es pot fer un anàlisi estadístic semblant al dels xifres clàssics, perquè els bytes més freqüents del text solen ser els mateixos, per exemple l'espai o les lletres més comunes.

5 Ataquem el nostre propi RSA

5.1 Construcció del diccionari

En el nostre sistema cada byte es xifra de manera independent, de manera que només existeixen 256 missatges possibles ($m = 0, 1, \dots, 255$). Qualsevol persona que conegui la clau pública (n, e) pot xifrar ella mateixa aquests 256 valors i guardar-los en un diccionari invers ($\text{diccionari}[c] = m$)

Ho hem fet amb la clau de l'apartat 3.3 ($p = 1009, q = 1013, n = 1022117, e = 65537$) i amb la nostra pròpia funció xifra. Després hem xifrat el text "La criptografia és divertida!" i hem consultat el diccionari per a cada enter del criptograma. Hem recuperat el text complet correctament **sense utilitzar la clau privada**.

5.2 Anàlisi de l'atac

1. **Heu pogut recuperar el missatge?**
Sí, el text recuperat és idèntic a l'original.
2. **Heu necessitat factoritzar n ?**
No. Mai hem calculat p ni q a partir de n .
3. **Heu necessitat calcular d ?**
No. La funció d'atac només rep (n, e) .
4. **Quina informació secreta heu necessitat conèixer?**
Cap. Només hem utilitzat (n, e) , que és la clau pública.
5. **Per què la clau pública és suficient per construir el diccionari?**
Perquè xifrar només requereix (n, e) , i la clau pública és, per definició, coneguda per tothom. Com que RSA és determinista i, per a una clau vàlida, cada $m \in [0, n)$

té un únic criptograma, el diccionari inverteix el xifratge de manera exacta: cada criptograma identifica un únic byte.

6. Què passaria si l'espai de missatges fos immens?

El diccionari seria impossible de construir. Amb blocs de 2048 bits hi hauria aproximadament 2^{2048} missatges possibles, molt més del que es pot calcular o emmagatzemar. L'atac només funciona perquè l'espai de missatges és diminut. Cal precisar que, si el missatge és molt previsible (per exemple un PIN o una frase típica), l'atacant pot provar només els candidats probables, encara que l'espai teòric sigui gran.

Repte

No hem trencat la teoria matemàtica de RSA: n'hem fet un ús inadequat.

La seguretat de RSA es basa en el fet que és computacionalment inviable invertir l'operació $m \rightarrow m^e \bmod n$ sense conèixer d (o la factorització de n) quan m pot ser qualsevol valor d'un espai enorme. Nosaltres no hem invertit aquesta funció: l'hem calculat per als 256 candidats i hem comparat resultats, és a dir, una cerca exhaustiva. El punt feble no és RSA, sinó l'esquema que hem construït a sobre: xifrem un sol byte cada vegada i de manera determinista, així que l'espai de missatges és trivial i el mateix byte dona sempre el mateix criptograma. Per això les implementacions reals afegeixen padding aleatori abans d'aplicar RSA, tal com hem observat amb OpenSSL a l'apartat 7.3.

6 Blocs de bytes

6.1 Experiment

Hem convertit el bloc b"Hola" a un enter amb `int.from_bytes(bloc, byteorder="big")` i hem obtingut 1215261793. Amb `m.to_bytes(4, byteorder="big")` hem recuperat els quatre bytes originals b"Hola".

Els bytes de "Hola" són [72,111,108,97], i l'enter obtingut és:

$$72 \cdot 256^3 + 111 \cdot 256^2 + 108 \cdot 256 + 97 = 1215261793$$

En general, uns bytes $b_0, b_1, \dots, b_{k-1}$ es poden interpretar com les xifres d'un nombre escrit en base 256:

$$m = b_0 \cdot 256^{k-1} + b_1 \cdot 256^{k-2} + \dots + b_{k-1}$$

Això és exactament el mateix que fem amb un nombre decimal, però amb base 256 en lloc de 10. Un byte pot prendre 256 valors (0–255), que són precisament les 256 “xifres” possibles, i cada posició té com a pes una potència de 256. Com que l'escriptura en una base és única, la conversió és reversible. Només cal conèixer la longitud k del bloc, ja que, si no, es perdrien els zeros inicials.

6.2 Quants bytes caben?

RSA exigeix $m < n$. Un bloc de k bytes pot representar valors fins a $256^k - 1$, així que busquem el màxim k tal que $256^k < n$.

1. Clau petita de l'exemple de classe ($n = 3233$):

$$256^1 = 256 < 3233 < 65536 = 256^2$$

per tant $k = 1$. Amb aquesta clau agrupar bytes no aporta res, perquè continuem xifrant un byte cada vegada. (Amb la nostra clau, $n = 1022117$, obtenim $k = 2$.)

2. **n d'aproximadament 2048 bits:**

$256^k < 2^{2048}$ equival a $2^{8k} < 2^{2048}$, és a dir $8k < 2048 \Rightarrow k < 256$, per tant $k = 255$ bytes (2040 bits).

3. **Agrupar bytes elimina per si sol els problemes observats?**

No. Agrupar bytes redueix el nombre d'operacions RSA i fa que el diccionari de l'apartat 5 sigui molt més gran (256^k candidats), però RSA continua sent determinista: el mateix bloc dona sempre el mateix criptograma, de manera que els blocs repetits segueixen sent visibles. A més, si el missatge és previsible, l'atacant pot continuar provant només els candidats més probables. La solució real és el padding aleatori.

7 RSA amb OpenSSL

Hem comparat la nostra implementació amb una biblioteca criptogràfica real.

7.1 Generació de les claus

En executar les comandes que es demanen a la pràctica i generant una clau de prova veiem clarament els diferents elements pertinents a l'RSA:

A la clau privada:

Mòdul (n)	Exponent privat (d)	Els dos primers (p, q)	Exponent públic (e)
modulus: 00:8a:aa:ab:31:9c:97:94:af:9d:e2:0d:9d:0a:2e: 49:a1:eb:0d:79:f5:de:87:ca:03:01:4a:e7:1f:18: ab:eb:0c:78:d3:9e:37:13:3c:8b:0d:34:f3:b4:0b: d9:1e:65:df:87:69:fc:ca:94:b8:99:86:bb:5d:97: c8:c4:99:aa:24:7c:09:10:99:af:80:b4:af:2d:32: 00:cb:35:e8:ef:65:45:e7:ed:5e:da:c5:e1:f3:35: 7c:3a:97:da:6c:96:5a:2f:3d:1e:fc:f3:0e:22:00: 26:5f:21:a9:4c:ed:dc:7d:47:20:7b:7b:18:11:28: ec:1c:f8:85:2a:0c:14:04:5b:0f:55:5a:f0:06:a1: bb:6c:ae:58:87:dc:a4:d9:0e:74:81:fc:da:8e:f6: a9:9c:94:2a:75:a5:61:68:ba:dc:34:c2:46:52:c8: 5d:df:48:6e:28:93:ef:7a:63:a3:8f:52:dc:d0:f2: 96:2d:0d:65:34:c6:98:33:db:93:2b:31:57:68:07: 21:51:d6:df:7b:78:f9:0f:1a:36:26:db:1d:bf:ed: e5:d0:07:01:2e:63:ef:6b:6f:c1:54:13:87:71:c0: eb:39:02:9f:78:01:10:99:26:dd:22:c1:d1:70:5c: 5a:55:05:39:85:67:26:39:75:34:da:86:4f:16:ef: f5:7f	privateExponent: 3d:6f:52:cc:76:a7:b3:a0:c0:ce:d3:a0:35:4f:ff: a0:64:9e:c2:5f:7b:99:66:97:41:3d:d9:b4:ed:b7: d1:39:e3:d8:41:da:fd:c3:4f:89:e8:e1:65:ed:ef: e2:27:59:4a:c9:da:cd:37:e9:b7:5f:39:68:c4:64: 17:32:4b:6e:c1:a3:54:7e:fa:82:07:cb:2a:b7:62: 21:a5:ff:4e:fc:eb:e5:4e:c5:bd:20:79:b4:42:88: 88:ff:19:2f:0d:c7:e3:f0:ec:e7:3d:9a:0f:d7:48: 28:d2:a5:8a:e7:41:4e:21:b2:92:f2:9c:dd:91:67: 56:88:86:9a:f3:e1:f4:71:6a:93:d5:54:30:13:59: 4c:ce:e7:f9:f8:24:4d:9d:fa:ce:1b:37:cb:c5:1d: f3:c0:87:44:cb:36:84:fb:bb:86:4a:ae:16:d9:3d: e3:18:e0:36:c1:0f:3f:c4:5a:45:97:df:dc:7c:5d: b1:37:28:08:f5:d8:4f:f9:6f:7a:a1:85:a1:f0:a1: ab:3b:b2:56:8a:c9:8d:f4:3b:9c:3d:21:c4:94:e0: 97:26:9d:91:35:a8:71:65:56:bd:58:2c:e1:dc:b6: 30:f0:eb:b6:37:e9:c7:4d:a8:ff:08:72:b3:b7:35: f5:b7:57:24:a5:ca:fb:b9:33:6d:86:80:eb:54:1e: 8d	prime1: 00:c0:60:0c:56:f0:5e:dd:34:3b:db:a0:5a:94:45: 93:83:1d:dc:7d:38:40:0c:3c:07:c3:2c:07:0a:88: fb:27:4b:88:1c:79:2a:76:2a:b8:60:88:62:68:75: 1f:cf:75:4e:af:42:28:67:9f:bd:41:66:a6:b9:85: ec:57:38:1c:24:b5:65:03:94:67:f6:c9:6d:fd:8e: e5:86:a7:44:65:2b:5a:cd:04:26:9c:e4:a0:85:da: b1:7f:e3:53:22:e8:6b:b5:90:ef:2b:cc:80:3c:f9: d6:6b:ab:10:bb:21:e3:3a:a7:23:98:8a:4c:65:5e: ec:af:75:c4:28:4c:0f:4f:83 prime2: 00:b8:07:3f:71:16:04:b7:14:dd:78:88:44:9d:1b: ed:2b:69:e6:b6:63:70:ef:05:94:d3:e6:18:3f:13: 65:39:69:fc:60:5e:53:03:5f:b4:02:80:91:d3:0c: 9f:25:ea:e3:a7:38:d0:b2:25:76:1c:90:c8:5f:bd: c2:2a:e4:93:23:a2:77:21:01:fe:6a:35:29:25:f2: 76:e9:8d:85:84:f7:08:ec:47:b8:44:dd:db:65:a8: ba:0e:c1:67:65:fe:cc:54:12:b9:11:cd:63:b4:6e: 56:65:76:37:26:18:91:6a:b8:37:66:5b:ae:08:44: ff:b4:84:71:31:1a:ef:05:55	publicExponent: 65537 (0x10001)

En canvi a la clau pública (n, e)

```

=== CLAU PÚBLICA ===
Public-Key: (2048 bit)
Modulus:
00:8a:aa:ab:31:9c:97:94:af:9d:e2:0d:9d:0a:2e:
49:a1:eb:0d:79:f5:de:87:ca:03:01:4a:e7:1f:18:
ab:eb:0c:78:d3:9e:37:13:3c:8b:0d:34:f3:b4:0b:
d9:1e:65:df:87:69:fc:ca:94:b8:99:86:bb:5d:97:
c8:c4:99:aa:24:7c:09:10:99:af:80:b4:af:2d:32:
00:cb:35:e8:ef:65:45:e7:ed:5e:da:c5:e1:f3:35:
7c:3a:97:da:6c:96:5a:2f:3d:1e:fc:f3:0e:22:00:
26:5f:21:a9:4c:ed:dc:7d:47:20:7b:7b:18:11:28:
ec:1c:f8:85:2a:0c:14:04:5b:0f:55:5a:f0:06:a1:
bb:6c:ae:58:87:dc:a4:d9:0e:74:81:fc:da:8e:f6:
a9:9c:94:2a:75:a5:61:68:ba:dc:34:c2:46:52:c8:
5d:df:48:6e:28:93:ef:7a:63:a3:8f:52:dc:d0:f2:
96:2d:0d:65:34:c6:98:33:db:93:2b:31:57:68:07:
21:51:d6:df:7b:78:f9:0f:1a:36:26:db:1d:bf:ed:
e5:d0:07:01:2e:63:ef:6b:6f:c1:54:13:87:71:c0:
eb:39:02:9f:78:01:10:99:26:dd:22:c1:d1:70:5c:
5a:55:05:39:85:67:26:39:75:34:da:86:4f:16:ef:
f5:7f
Exponent: 65537 (0x10001)

```

Comparació amb la nostra implementació

A la sortida d'OpenSSL hi trobem els mateixos elements que vam construir als apartats 3.1 i 3.3: el mòdul n , l'exponent públic e , l'exponent privat d i els dos primers p i q . La relació entre ells és la mateixa: $n = p \cdot q$, la clau pública és (n, e) i la privada és (n, d) , amb d l'invers modular d' e respecte a l'ordre del grup multiplicatiu $(\varphi(n))$.

Les diferències són sobretot de mida:

- **Mida:** nosaltres vam utilitzar $p = 1009$ i $q = 1013$, que donen $n = 1022117$ (uns 20 bits) i $d = 832193$. OpenSSL genera dos primers de 1024 bits, un mòdul de 2048 bits i un d d'aquesta mateixa mida, impossible de calcular a mà.
- **Exponent públic i generació:** en tots dos casos $e = 65537$ (0x10001). La diferència és que nosaltres vam triar p i q d'una llista donada, mentre que OpenSSL els genera aleatòriament i comprova que són primers.

La mida és el que marca la seguretat. Amb la nostra $n = 1022117$ es pot factoritzar de manera immediata per divisió, i amb p i q es calcula d . Amb una n de 2048 bits no es coneix cap manera de factoritzar-la en un temps raonable, i per això es pot publicar (n, e) sense revelar d .

7.2 Xifratge amb OpenSSL

Per a aquest experiment hem utilitzat la clau de 2048 bits publica_bob.pem / privada_bob.pem. Hem creat el fitxer missatge.txt amb el text "Missatge secret", l'hem xifrat amb la clau pública i l'hem desxifrat amb la clau privada:

```
openssl pkeyutl -encrypt -pubin -inkey claus/publica_bob.pem \  
-in missatge.txt -out missatge.enc \  
-pkeyopt rsa_padding_mode:pkcs1
```

```
openssl pkeyutl -decrypt -inkey claus/privada_bob.pem \  
-in missatge.enc -out recuperat.txt \  
-pkeyopt rsa_padding_mode:pkcs1
```

Amb `cat recuperat.txt` hem obtingut "Missatge secret", idèntic a l'original. El criptograma ocupa 256 bytes (2048 bits) tot i que el missatge és molt curt. Cal destacar que OpenSSL no aplica RSA directament al missatge, sinó que primer el transforma mitjançant el mecanisme de padding, a diferència de la nostra implementació.

7.3 El mateix missatge dues vegades

```
sha256sum missatge1.enc missatge2.enc  
cmp missatge1.enc missatge2.enc  
1c73d86a033e11fa11cab79b510672b1e6f48618bfee682c421601142dbbadd1 missatge1.en  
c  
c0d38ad1a569fc32218aebcb64e22a5817250a667a53342ae8aa31e54f9d942 missatge2.en  
c  
missatge1.enc missatge2.enc differ: byte 1, line 1
```

1. **Els dos criptogrames obtinguts són completament diferents**, com confirmen els hash SHA-256 resultants.
2. **Amb el nostre programa el mateix missatge donava sempre el mateix criptograma.** Per exemple, quan vam xifrar "La criptografia és divertida!" cada aparició de la lletra "a" es va xifrar sempre com 745232. Això passa perquè el nostre xifratge és determinista, sense cap component aleatori.
3. **El missatge i la clau pública eren idèntics en les dues execucions.** L'única cosa que ha canviat és la informació aleatòria que el padding introdueix cada vegada que es xifra.

4. **No. RSA és una funció determinista:** el mateix missatge amb la mateixa clau sempre produeix el mateix resultat. Com que els criptogrames són diferents, queda demostrat que OpenSSL no aplica $c = m^e \bmod n$ directament al missatge original.
5. **El padding afegeix bytes aleatoris al missatge abans d'aplicar RSA.** Això fa que el bloc que entra a l'operació matemàtica sigui diferent en cada execució i, per tant, el criptograma resultant també ho sigui.

8 Alice i Bob

Hem treballat amb una altra parella per intercanviar missatges xifrats.

8.1 Generació i intercanvi de claus

Hem generat les claus RSA de 2048 bits amb OpenSSL. Nosaltres, com a Bob, hem creat la nostra clau privada (`privada_bob.pem`) i n'hem extret la clau pública (`publica_bob.pem`). L'altra parella, com a Alice, ha fet el mateix procés.

L'intercanvi ha consistit únicament en les claus públiques: nosaltres hem lliurat `publica_bob.pem` a Alice i Alice ens ha lliurat `publica_alice.pem`. Les claus privades no s'han compartit en cap moment.

8.2 Alice envia un missatge a Bob

Alice ha escrit un missatge que nosaltres desconeixem, l'ha xifrat utilitzant la nostra clau pública (`publica_bob.pem`) i ens ha enviat el fitxer xifrat (`missatge_alice.enc`).

Nosaltres hem recuperat el missatge original utilitzant la nostra clau privada (`privada_bob.pem`):

```
openssl pkeyutl -decrypt \  
-inkey claus/privada_bob.pem \  
-in criptogrames/missatge_alice.enc \  
-out criptogrames/missatge_recuperat_alice.txt \  
-pkeyopt rsa_padding_mode:pkcs1
```

Respostes a les preguntes

1. **Quina clau ha utilitzat Alice?**
Alice ha utilitzat la clau pública de Bob (`publica_bob.pem`) per xifrar el missatge. Aquesta és l'única clau que Alice necessita i que Bob li ha proporcionat prèviament.
2. **Quina clau necessita Bob per recuperar el missatge?**
Bob necessita la seva pròpia clau privada (`privada_bob.pem`). Aquesta clau és l'única que permet desxifrar un missatge que ha estat xifrat amb la clau pública corresponent.
3. **Alice necessita conèixer la clau privada de Bob?**
No. Alice només necessita la clau pública de Bob. Precisament aquest és l'avantatge del xifratge asimètric: l'emissor pot xifrar sense conèixer cap informació secreta del receptor.
4. **Una tercera persona pot conèixer la clau pública de Bob?**
Sí. La clau pública, com el seu nom indica, és pública i es pot distribuir lliurement.

Qualsevol persona pot tenir-la sense que això comprometi la seguretat del sistema.

5. **Una tercera persona que només tingui la clau pública pot desxifrar el missatge?**

No. La clau pública en aquest esquema només permet xifrar. Per desxifrar cal la clau privada, que només posseeix Bob. La seguretat de RSA es basa en el fet que, tot i coneixent la clau pública (n, e) , és computacionalment inviable obtenir la clau privada (d) sense poder factoritzar n en els seus dos factors primers p i q .

8.3 Canvi de papers

Hem repetit l'experiment en sentit contrari. Nosaltres (Bob) hem escrit un missatge original (frase_original_bob.txt) i l'hem xifrat amb la clau pública d'Alice (publica_alice.pem):

```
openssl pkeyutl -encrypt \
  -pubin \
  -inkey claus/publica_alice.pem \
  -in criptogrames/frase_original_bob.txt \
  -out criptogrames/text_xifrat_per_alice.enc \
  -pkeyopt rsa_padding_mode:pkcs1
```

Hem enviat el fitxer xifrat (text_xifrat_per_alice.enc) a Alice, que l'ha pogut desxifrar amb la seva clau privada.

8.4 Esquema final

Alice —(publica_bob.pem)→ missatge_alice.enc —(privada_bob.pem)→ Bob

Bob —(publica_alice.pem)→ text_xifrat_per_alice.enc —(privada_alice.pem)→ Alice

Aquest sistema funciona gràcies a la propietat fonamental del xifratge asimètric: la clau que xifra (pública) i la que desxifra (privada) són diferents. Això permet que la clau de xifratge sigui pública sense comprometre la seguretat, ja que no és possible derivar la clau privada a partir de la pública en un temps raonable. D'aquesta manera, qualsevol persona pot enviar un missatge xifrat al receptor sense necessitat d'haver acordat prèviament un secret compartit.

9 Auditoria d'un programa generat per IA

Hem reproduït el programa de l'enunciat a auditoria_ia.py i hem construït casos concrets per comprovar-ne cada error. Un detall previ és que el programa de la IA crida invers_modular sense definir-la enlloc, per això l'hem hagut d'afegir nosaltres per poder executar-lo.

9.1 Generació de les claus

La funció genera_claus de la IA no fa cap comprovació. Accepta qualsevol p , q i e i els multiplica, i això produeix claus incorrectes o inutilitzables.

$p = q$. Amb $p = q = 13$ i $e = 5$ obtenim $n = 169$ i la fórmula $(p - 1)(q - 1)$ dona 144, però aquesta fórmula només és vàlida quan els dos primers són diferents. La funció calcula $d = 29$ i llavors xifrar 7 dona 76 i desxifrar 76 dona 33 en lloc de 7. El motiu és que quan $p = q$ la

φ real de $n = p^2$ és $p^2 - p = 156$ i no 144, així que d no és l'invers correcte. A més, si n és un quadrat perfecte es pot factoritzar de seguida.

p no primer. Amb $p = 51$ ($51 = 3 \cdot 17$) i $q = 61$ obtenim $n = 3111$. La fórmula calcula $50 \cdot 60 = 3000$, però la φ real és $\varphi(51) \cdot \varphi(61) = 32 \cdot 60 = 1920$. El $d = 2143$ obtingut és l'invers de 7 mòdul 3000 i no mòdul 1920, així que xifrar 10 dona 1246 i desxifrar dona 925 en lloc de 10.

q no primer. Amb $p = 1009$ i $q = 1024$ (2^{10}) obtenim $n = 1033216$. La fórmula dona $1008 \cdot 1023 = 1031184$, però la φ real és $1008 \cdot \varphi(1024) = 1008 \cdot 512 = 516096$. El $d = 909809$ no funciona, i xifrar 42 dona 331776 que es desxifra com 829440.

e no coprimar amb $\varphi(n)$. Amb $p = 61$ i $q = 53$ tenim $\varphi(n) = 3120$, i amb $e = 13$ el $\text{mcd}(13, 3120) = 13$ no és 1. L'invers no existeix, `invers_modular` retorna `None` i el programa entrega igualment la clau `((3233, 13), (3233, None))` sense cap avís. La clau privada no existeix i el programa no ho detecta.

La conclusió de l'apartat 9.1 és que qualsevol d'aquestes situacions produeix una clau que no serveix per desxifrar i el programa no se'n queixa. La nostra `genera_claus`, en canvi, rebutja els quatre casos i retorna `None`.

9.2 Xifratge del text

- Quina diferència hi ha entre `ord(c)` i `c.encode("utf-8")`?**
`ord(c)` retorna el punt de codi Unicode del caràcter, un únic nombre. `c.encode("utf-8")` retorna la seqüència de bytes que representa aquest caràcter. Per als caràcters ASCII coincideixen, per als altres no.
- Què passa amb un caràcter com `ú` o `ç`?**
`ord("ú")` és 250 mentre que UTF-8 dona `[195, 186]`, dos bytes. `ord("ç")` és 231 i UTF-8 dona `[195, 167]`. El programa xifra un sol nombre per caràcter en lloc dels bytes reals. En desxifrar obtindrem els punts de codi i, en reconstruir el text, `bytes(...).decode("utf-8")` no reproduirà els caràcters originals. Els textos amb accents no es recuperen correctament.
- Es comprova que el valor que entra a RSA compleixi $m < n$?**
 No. Si apareix un caràcter amb codi més gran que n , per exemple `ord('€') = 8364` amb $n = 3233$, el `pow` redueix silenciosament la base, $8364 \bmod 3233 = 1898$, i el resultat és igual que si xifréssim 1898. En desxifrar obtenim 1898 i no 8364, perquè s'ha perdut informació sense cap error.
- Què passa si apareixen dues lletres iguals?**
 Es xifren igual i es repeteixen al criptograma. Amb la clau de l'exemple, `AAAAAAAAAA` dona deu vegades 2790, perquè A és 65 i $65^{17} \bmod 3233 = 2790$. A `MISSISSIPPI` es veuen les repeticions directament al criptograma.
- El resultat s'assembla més al vostre primer programa o al comportament observat amb OpenSSL?**
 S'assembla molt més al nostre primer programa, byte a byte i determinista. No s'assembla gens a OpenSSL, que amb el padding PKCS#1 introdueix aleatorietat i canvia el criptograma cada vegada, tal com hem vist a l'apartat 7.3.

9.3 Conclusió de l'auditoria

Les tres afirmacions de l'enunciat no són equivalents.

1. **El programa executa una operació matemàtica relacionada amb RSA.**
Sí. Calcula $m^e \bmod n$ i $c^d \bmod n$, que són les operacions centrals de RSA.
2. **El programa implementa correctament l'algorisme matemàtic de RSA.**
No. La generació de claus no comprova que p i q siguin primers i diferents ni que e sigui coprimer amb $\varphi(n)$, i quan p o q no són primers la φ calculada és incorrecta i el desxifratge falla. A més xifra punts de codi en lloc de bytes i no comprova $m < n$.
3. **El programa constitueix un sistema de xifratge segur.**
No. A més dels errors anteriors, el xifratge és determinista i es fa byte a byte, així que es veuen les repeticions i funciona l'atac per diccionari de l'apartat 5. Tampoc té padding ni un espai de missatges prou gran.

Que un programa faci servir l'operació de RSA no vol dir que l'implementi correctament, i implementar-la correctament no vol dir que el sistema sigui segur. La seguretat depèn de tot l'esquema (claus prou grans, padding aleatori i ús adequat) i no només de la fórmula. Per això l'afirmació de la IA, que el programa permet generar claus i xifrar qualsevol text de manera segura, és falsa.

14 Conclusions

Hem construït RSA completament, des de les eines matemàtiques fins a la comunicació entre dues parelles, i hem pogut comprovar tant quan funciona com quan no.

- L'exponenciació modular ràpida permet calcular $a^e \bmod n$ amb un nombre de multiplicacions proporcional al nombre de bits de e , i això fa factible RSA amb exponents grans.
- La implementació matemàtica de RSA funciona. Hem generat claus i hem xifrat i desxifrat correctament missatges complets.
- El seu ús ingenu no és segur. El xifratge byte a byte deixa veure les repeticions del text i l'espai de missatges de 256 valors permet recuperar el missatge amb un simple diccionari, sense factoritzar n ni calcular d .
- Agrupar bytes en blocs augmenta l'espai de missatges però no acaba amb el problema, perquè RSA continua sent determinista.
- Les implementacions reals resolen aquests problemes amb padding aleatori i claus grans. Hem observat que OpenSSL produeix criptogrames diferents per al mateix missatge i que una clau de 2048 bits no es pot factoritzar.
- L'esquema asimètric permet que Alice i Bob es comuniquin només compartint claus públiques, sense haver d'acordar cap secret per endavant.
- Un programa generat per IA pot semblar correcte i no ser-ho. Cal auditar-lo amb casos concrets, perquè el fet que una operació estigui relacionada amb RSA no vol dir que la implementació sigui correcta ni que el sistema sigui segur.